

Statistical Data Science



07

Statistics is the study of collecting, analysing, and interpreting data to uncover patterns and trends. It provides essential tools for making informed decisions in various fields, from business to healthcare.

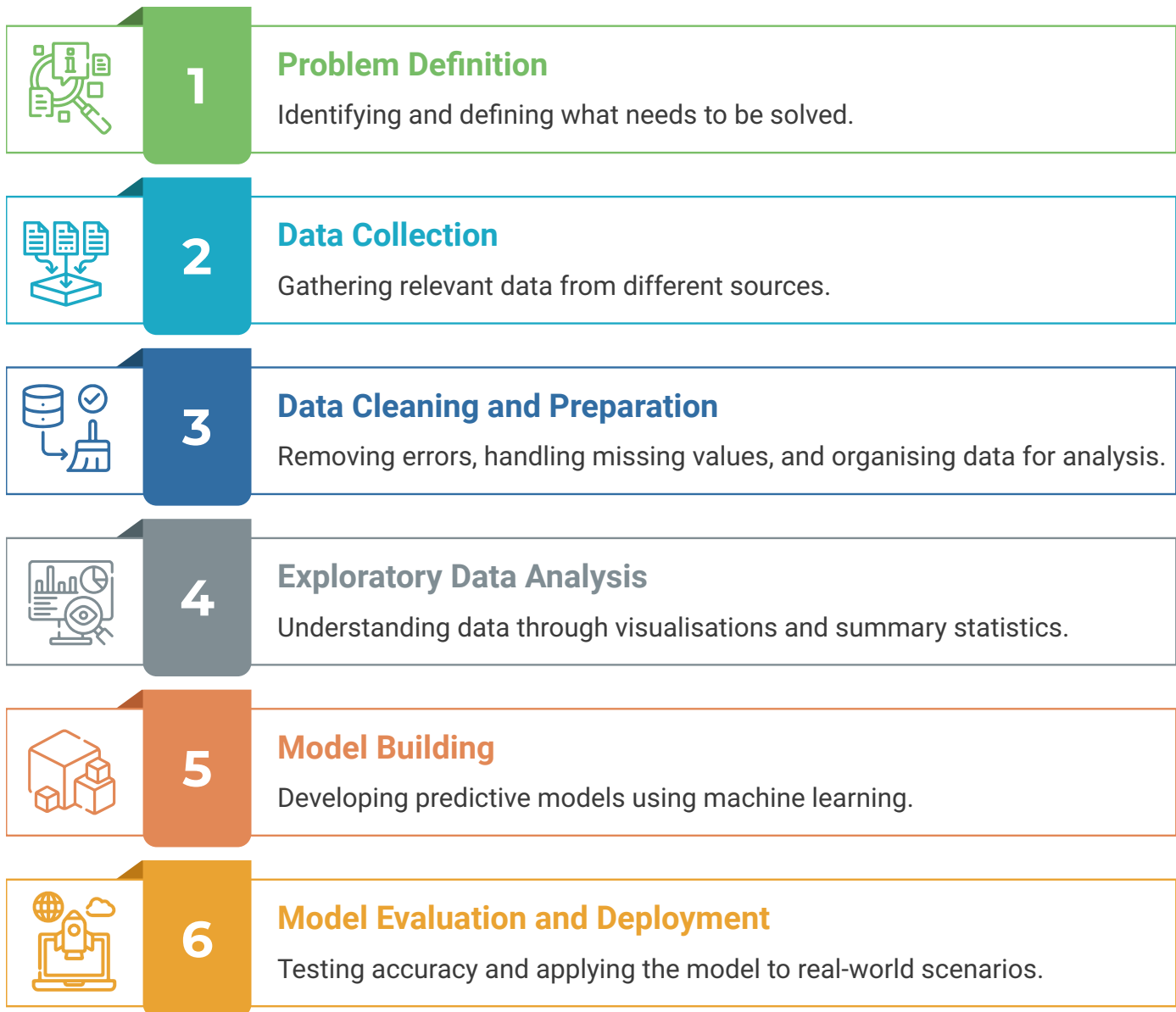


In today's data-driven world, decision-making in fields such as business, healthcare, and technology relies on extracting meaningful insights from data. To extract meaningful information from data, systematic methods are required to organise, analyse, and interpret information effectively. Data science, an interdisciplinary field, integrates statistics, mathematics, and programming to extract valuable information, identify patterns, and forecast trends, enabling strategic decision-making.

Among these, statistics plays a vital role in summarising data, identifying trends, and supporting reliable decision-making. In this lesson, we will explore key statistical methods used in data science for effective analysis.

Data Science Process Overview

The data science process consists of several key steps:



For complex projects, additional refinement steps may be required to enhance accuracy and ensure the model remains effective with new data over time.

Role of Statistics in Data Science

Statistics is the study of collecting, organising, analysing, and interpreting data to make informed decisions. It helps us understand patterns, trends, and relationships within data.

In data science, statistics plays a crucial role in summarising large datasets, identifying patterns, and making predictions. Essential statistical techniques—such as probability, hypothesis testing, and regression analysis—help ensure that data-driven decisions are accurate and reliable.

Without statistics, data science would lack the foundation to process and interpret data correctly.

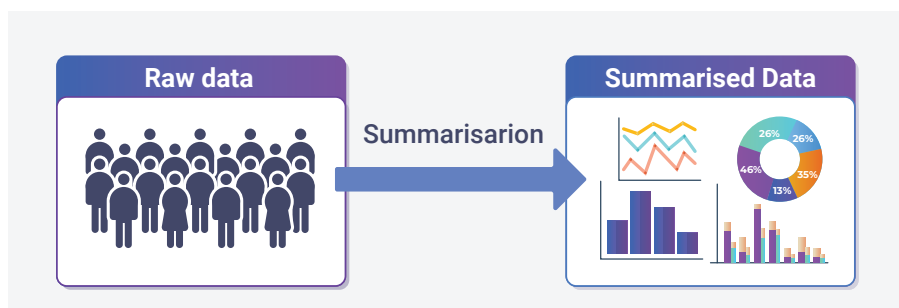


Types of Statistics

Statistics is broadly classified into two types: Descriptive Statistics and Inferential Statistics.

I. Descriptive Statistics

Descriptive statistics help summarise and present data in a meaningful way. It includes measures of central tendency (which show the centre of the data) and dispersion (which show the spread of data), helping us understand patterns and compare datasets easily.



a) Central Tendency: It refers to measures that indicate the centre or most common value in a dataset. There are three main measures of central tendency:

1. Mean (Average): The sum of all values divided by the number of values.

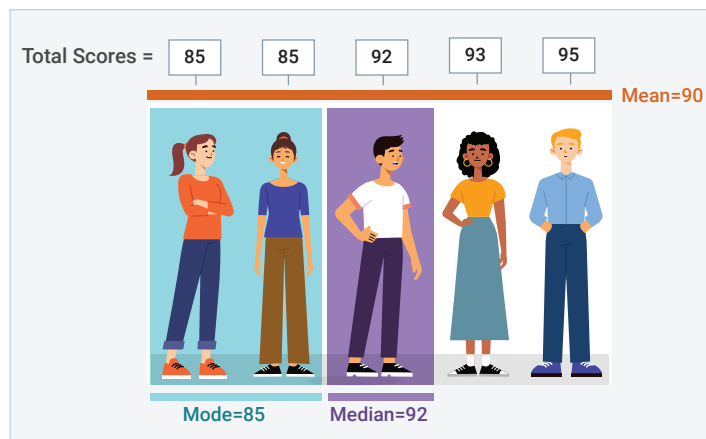
- Example: If students scored 85, 93, 85, 92 and 95 on a test, the mean is $(85 + 93 + 85 + 92 + 95) / 5 = 90$.

2. Median: The middle value in a sorted dataset.

- Example: In the scores [85, 85, 92, 93, 95], the median is 92.

3. Mode: The most frequently occurring value in the dataset.

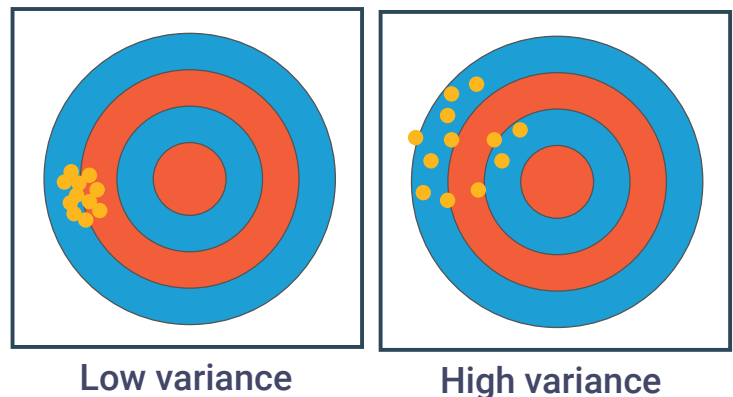
- Example: In the scores [85, 93, 85, 92, 95], the mode is 85 because it appears twice.



b) Dispersion: It measures how much the data values vary or spread out from the average (mean).

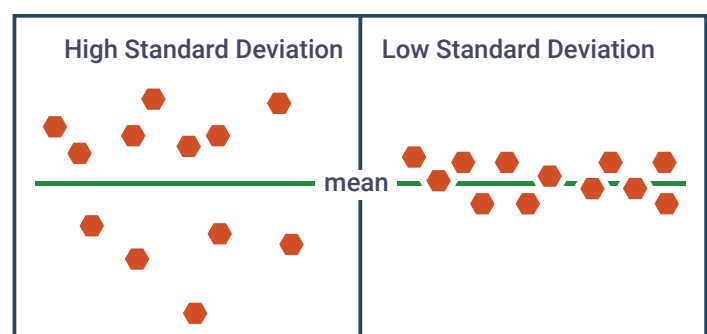
- 1. Variance:** Variance measures how far data points deviate from the mean. It is the average of the squared differences from the mean. A high variance means the data points are spread out, while a low variance means they are close to the mean. This helps measure data consistency.

- Example: If the heights of students in a class have a high variance, it means there is a significant difference in their heights.



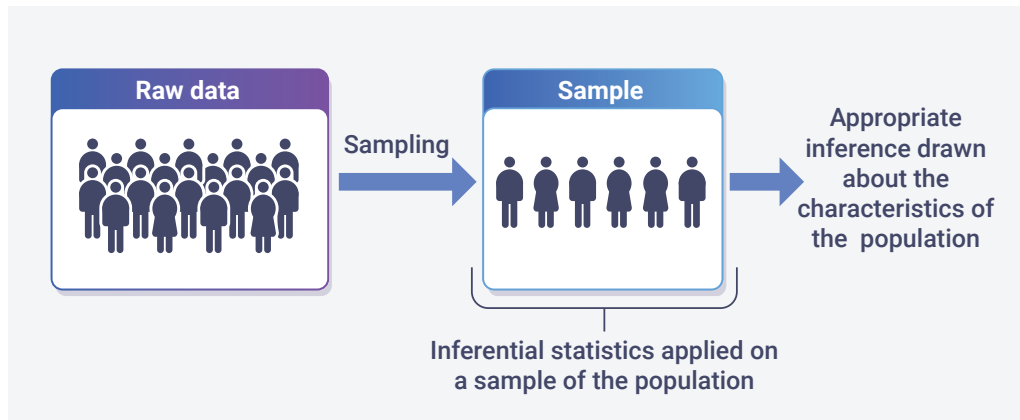
- 2. Standard Deviation (SD):** Standard Deviation (SD) is the square root of variance. It shows how much the data varies from the mean, making it easier to understand the spread and consistency of the data.

- Example: If test scores have a high SD, it means student performance varies widely. A low SD means most students scored close to the average.



II. Inferential Statistics

While descriptive statistics summarise data, inferential statistics allow us to make predictions and draw conclusions about a large group (population) based on a sample. Inferential statistics has two important aspects: Hypothesis Testing and Confidence Intervals.



a) Hypothesis Testing: A method to check if an assumption about a population is true by analysing sample data.

- **Example:** A teacher tests whether providing extra practice improves students' maths scores by comparing the results of two groups—one that receives extra practice and one that does not.

b) Confidence Intervals: A confidence interval is a range that likely contains the true population value, providing an estimate with a specified level of confidence. It helps quantify the uncertainty in sample estimates.

- **Example:** If a sample suggests that the average height of students is 5.5 feet $\pm$ 0.2 feet, it means we are 95% confident that the actual average height of all students falls between 5.3 and 5.7 feet.

Using concepts from both descriptive and inferential statistics, we can effectively study data, draw conclusions, and apply insights to real-world problems in data science.

Activity

Build a data science model to analyse how various features affect house prices.

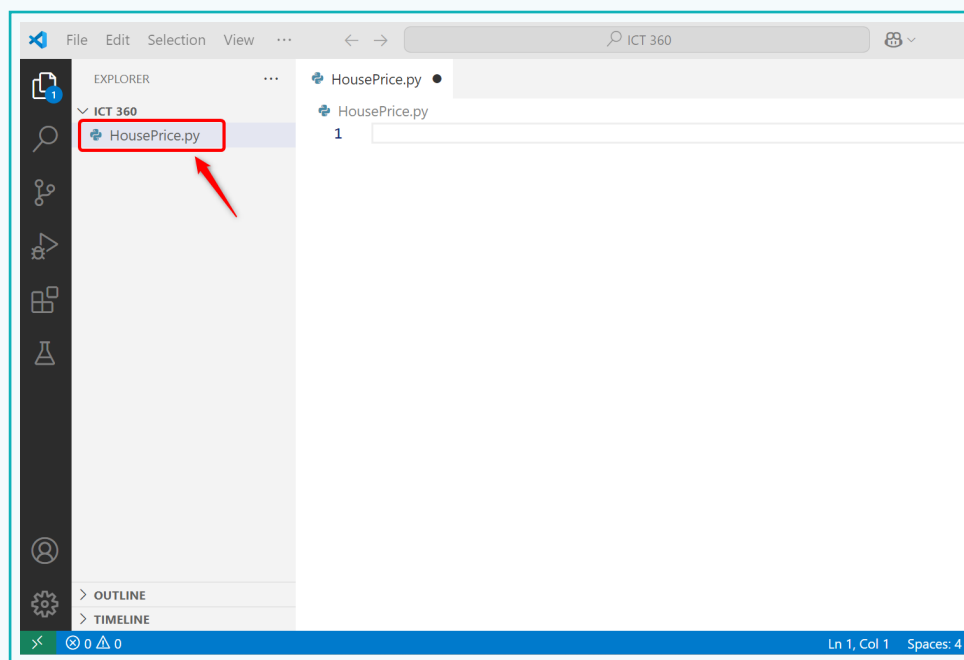
Part 1 - Define the Problem

- **Understand the Goal:** The goal is to analyse how various factors, such as location, number of bedrooms, and house size, influence house prices. Based on these features, we aim to build a model that can accurately predict house prices.
- **Specify the Objective:** The objective is to create a machine-learning model that predicts house prices based on given factors. This model will help buyers, sellers, and real estate agents make informed decisions regarding property pricing.

Part 2 - Collect the Data

1 Initial Setup

- Create a new folder in the desired location.
 - Open the VS Code editor, click the 'File' menu, and select 'Open Folder'. Locate the folder you created and click on 'Select Folder'.
 - Once the folder is displayed in the VS Code editor sidebar, create a new file named 'HousePrice.py' within it.



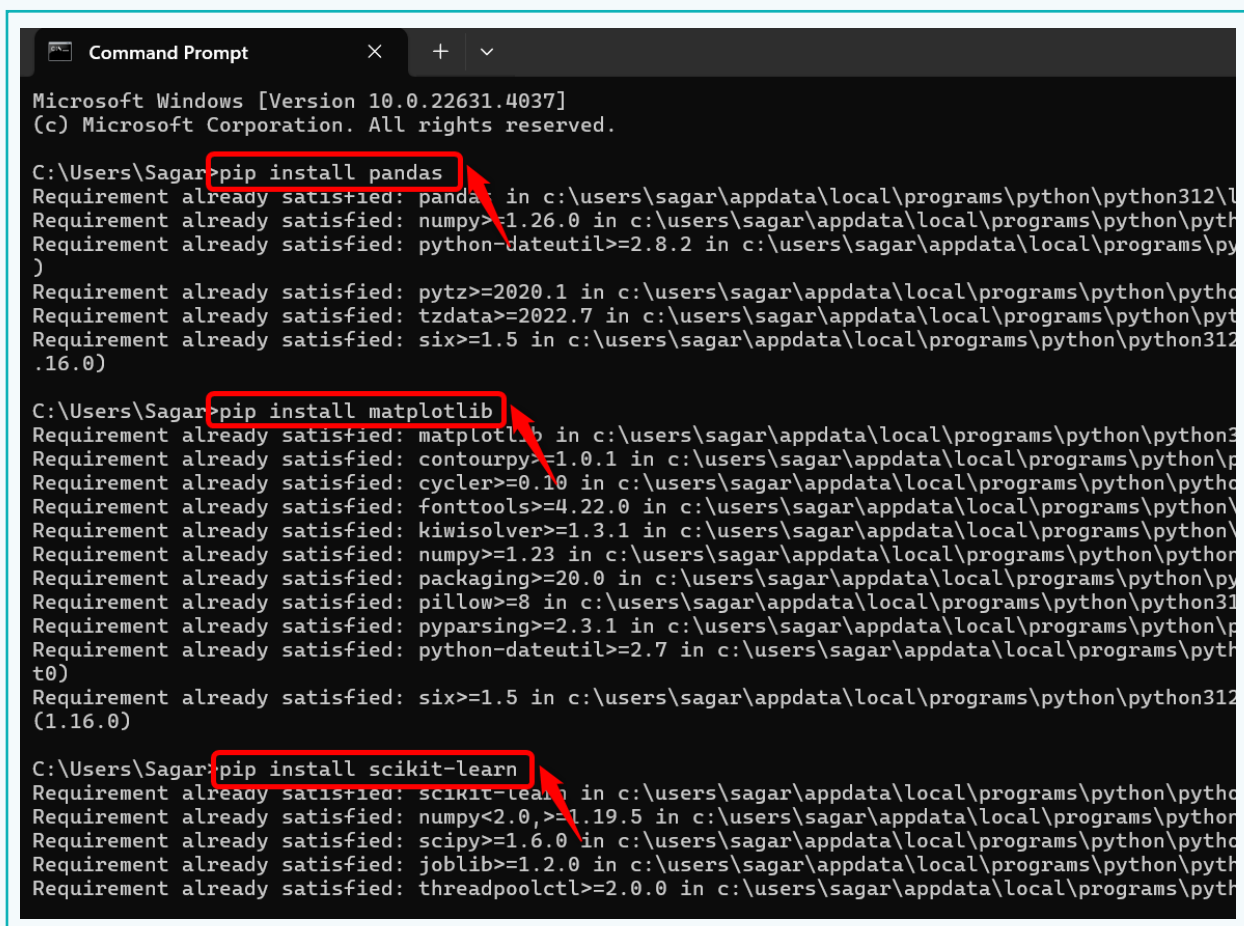
Activity

2 Install and Import Necessary Modules

First, we must install essential Python libraries that help with data manipulation, visualisation, and modelling.

- Pandas:** Helps handle and analyse structured data using DataFrames.
- Matplotlib:** Used for creating various types of visualisations.
- Scikit-learn:** Provides machine learning tools, including Linear Regression.

- Open the Command Prompt and run the following commands to install all three modules.



```
Microsoft Windows [Version 10.0.22631.4037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sagar>pip install pandas
Requirement already satisfied: pandas in c:\users\sagar\appdata\local\programs\python\python312\l
Requirement already satisfied: numpy>=1.26.0 in c:\users\sagar\appdata\local\programs\python\pyth
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\sagar\appdata\local\programs\pyth
)
Requirement already satisfied: pytz>=2020.1 in c:\users\sagar\appdata\local\programs\python\pytho
Requirement already satisfied: tzdata>=2022.7 in c:\users\sagar\appdata\local\programs\python\pyt
Requirement already satisfied: six>=1.5 in c:\users\sagar\appdata\local\programs\python\python312
.16.0)

C:\Users\Sagar>pip install matplotlib
Requirement already satisfied: matplotlib in c:\users\sagar\appdata\local\programs\python\python3
Requirement already satisfied: contourpy>=1.0.1 in c:\users\sagar\appdata\local\programs\python\p
Requirement already satisfied: cycler>=0.10 in c:\users\sagar\appdata\local\programs\python\pytho
Requirement already satisfied: fonttools>=4.22.0 in c:\users\sagar\appdata\local\programs\python\
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\sagar\appdata\local\programs\python\
Requirement already satisfied: numpy>=1.23 in c:\users\sagar\appdata\local\programs\python\pythor
Requirement already satisfied: packaging>=20.0 in c:\users\sagar\appdata\local\programs\python\py
Requirement already satisfied: pillow>=8 in c:\users\sagar\appdata\local\programs\python\python31
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\sagar\appdata\local\programs\python\p
Requirement already satisfied: python-dateutil>=2.7 in c:\users\sagar\appdata\local\programs\pyth
t0)
Requirement already satisfied: six>=1.5 in c:\users\sagar\appdata\local\programs\python\python312
(1.16.0)

C:\Users\Sagar>pip install scikit-learn
Requirement already satisfied: scikit-learn in c:\users\sagar\appdata\local\programs\python\pytho
Requirement already satisfied: numpy<2.0,>=1.19.5 in c:\users\sagar\appdata\local\programs\pythor
Requirement already satisfied: scipy>=1.6.0 in c:\users\sagar\appdata\local\programs\python\pytho
Requirement already satisfied: joblib>=1.2.0 in c:\users\sagar\appdata\local\programs\python\pyth
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\sagar\appdata\local\programs\pyth
```

Activity

- Import the libraries required for data handling, visualisation, and modelling in the Python file.
 - Import *pandas* with the alias 'pd' for efficient data manipulation.
 - Import *pyplot* from *matplotlib* with the alias 'plt' for creating plots.
 - Import *LinearRegression* from *scikit-learn* for building and training the regression models.

```
HousePrice.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4
5
```

3 Load the Dataset

We will use a publicly available dataset containing house price information, accessible from the following URL: <https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv>

- Save the website link as a string in the 'url' variable. Use the *pd.read_csv()* function to read the dataset and store it in a 'df' variable.

```
4 from sklearn.metrics import mean_absolute_error, r2_score
5
6 # Load dataset
7 url = 'https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv'
8 df = pd.read_csv(url)
9
10
```

Now, we will display the first few rows of the data.

- Use the *head()* function to display the first few rows of the DataFrame stored in 'df' to verify that the data has been loaded correctly.
 - Wrap it in the *print()* function to display the output.

```
8 df = pd.read_csv(url)
9
10 # Display first few rows
11 print(df.head())
12
13
```


Activity

- Execute the code to display the output.

```

PS :~> & ./.Users/ /AppData/Local/Programs/Python/Python312/python.exe "d:/ICT 360/HousePrice.py"
longitude latitude housing_median_age total_rooms total_bedrooms population households median_income m
0 -122.23 37.88 41.0 880.0 129.0 322.0 126.0 8.3252
1 -122.22 37.86 21.0 7099.0 1106.0 2401.0 1138.0 8.3014
2 -122.24 37.85 52.0 1467.0 190.0 496.0 177.0 7.2574
3 -122.25 37.85 52.0 1274.0 235.0 558.0 219.0 5.6431
4 -122.25 37.85 52.0 1627.0 280.0 565.0 259.0 3.8462
PS C:\Users\ >

```

Note:

This is a dataset of housing region. Each row represents a region with details about housing, income, and population.

- total_rooms: Number of all rooms in all houses in the region (unit: count).
- housing_median_age: Median age of houses (unit: years).
- median_income: Median household income (unit: value * 10,000, e.g. 4.2 ~ 42,000 currency units).

Part 3 - Prepare the Data

1 Check for Missing Values

Before proceeding, check if there are any missing values in the dataset to obtain precise results and eliminate any potential biases or inconsistencies.

For this, we will use the `isnull()` function.

- Use the `isnull()` function combined with `sum()` to check for missing values in the dataset stored in 'df'. This will show how many missing values exist in each column.
 - Wrap these functions in the `print()` function to display the output. Then, execute the code.

```

11 print(df.head())
12
13 # Check for missing values
14 print(df.isnull().sum())
15
16

```

Activity

Output

```
[5 rows x 10 columns]
longitude          0
latitude           0
housing_median_age 0
total_rooms         0
total_bedrooms     207
population         0
households         0
median_income      0
median_house_value 0
ocean_proximity    0
dtype: int64
PS D:\ICT 360>
```

We can observe that a few values (207) are missing in 'total_bedrooms'. In such cases, we can either fill them with appropriate values or remove the rows containing them.

2 Drop Missing Values

- Since missing values are present in the dataset, drop them from the data using `dropna()` function and save the cleaned data back into the 'df' variable.

```
14 print(df.isnull().sum())
15
16 # Remove rows with missing values
17 df = df.dropna()
18
```

Note:

After dropping the missing values, you can use the `isnull()` function again to confirm that all missing data has been removed.

Once the null values are removed, we can proceed to Exploratory Data Analysis (EDA).

Activity

Part 4 - Exploratory Data Analysis (EDA)

Now, let us visualise the relationships between house prices and different features such as median income, house age, and total rooms by plotting the information on a graph to understand the data better.

1 Create Scatter Plot

- Create scatter plots to explore the relationship between house price and other features. Use `plt.figure()` to create a new figure for plotting. Set the figure size to (12, 6) to provide enough space for the three plots.

```
17 df = df.dropna()
18
19 # EDA - Scatter Plots
20 plt.figure(figsize=(12, 6))
21
22
```

2 Create Subplot

Since we want to display and compare the relationships between all three house features versus house price (Median house value) in a single figure, we will use subplots.

- Create a Subplot for Median Income vs House Price**
 - Use `plt.subplot()` to define the layout of subplots. Set it to (1, 3, 1) to create the first subplot in a 1-row, 3-column grid, 1-first position.
 - Plot a scatter plot of Median income versus House price using the `plt.scatter()` function. Use `df['median_income']` to index the 'Median income' column and `df['median_house_value']` to index the 'House price' column from the DataFrame. Set `alpha=0.5` to make the points semi-transparent.
 - Add a title with `plt.title()`, x-axis label with `plt.xlabel()`, and y-axis label with `plt.ylabel()`.

```
21
22 # Median Income vs House Price
23 plt.subplot(1, 3, 1)
24 plt.scatter(df['median_income'], df['median_house_value'], alpha=0.5)
25 plt.title('Median Income vs House Price')
26 plt.xlabel('Median Income')
27 plt.ylabel('House Price')
28
29
```

Similarly, we will plot the scatter plots for other features.

Activity

- **Create a Subplot for House Age vs House Price**

- Use `plt.subplot()` to position the subplot in the second slot within the grid layout.
- Plot a scatter plot of House Age versus House Price using `plt.scatter()`.
- Set the title and axis labels as shown.

```
28
29 # House Age vs House Price
30 plt.subplot(1, 3, 2)
31 plt.scatter(df['housing_median_age'], df['median_house_value'], alpha=0.5)
32 plt.title('House Age vs House Price')
33 plt.xlabel('House Age')
34 plt.ylabel('House Price')
35
36
```

- **Create a Subplot for Total rooms vs House price**

- Use `plt.subplot()` to position the subplot in the third slot within the grid layout.
- Plot a scatter plot of Total Rooms versus House Price using `plt.scatter()`.
- Set the title and axis labels as shown.

```
36 # Total Rooms vs House Price
37 plt.subplot(1, 3, 3)
38 plt.scatter(df['total_rooms'], df['median_house_value'], alpha=0.5)
39 plt.title('Total Rooms vs House Price')
40 plt.xlabel('Total Rooms')
41 plt.ylabel('House Price')
42
```

- **Display the Scatter plots**

- Use `plt.tight_layout()` to adjust the spacing between subplots for a clean, non-overlapping layout.
- Display all the plots with `plt.show()`.

```
41 plt.ylabel('House Price')
42
43 plt.tight_layout()
44 plt.show()
45
46
```

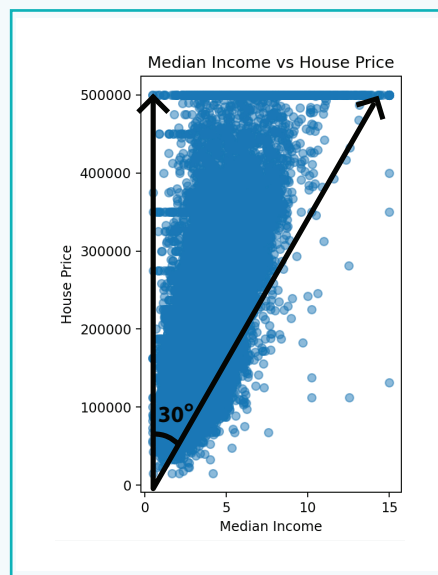
Activity

- Execute the code to display the output.



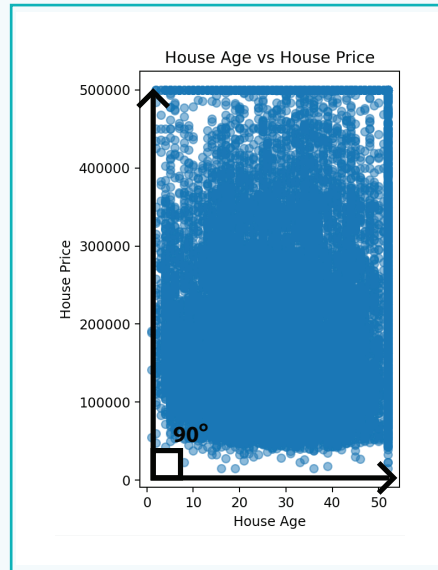
Findings from the Scatter Plots

As house prices increase, median income also increases, showing a clear relationship between the two. The data points spread in a pattern that, if measured, aligns at 30 degrees. This makes it visible that higher income leads to higher house prices, forming a noticeable trend.

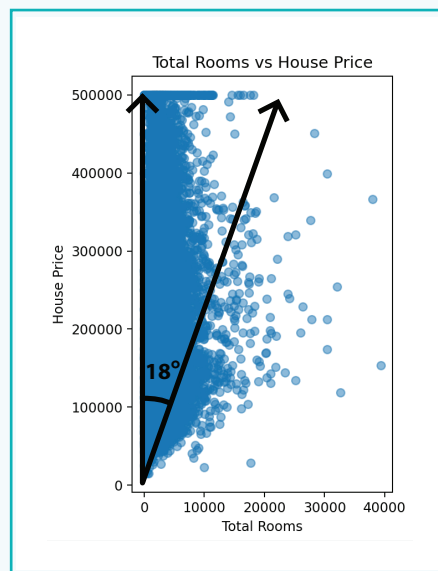


Activity

As house age increases, house prices remain scattered, forming a 90-degree spread. This shows that there is no clear pattern between house age and price. Prices vary widely for new and old houses, indicating that age does not impact house prices.



As the total number of rooms increases, the house price also increases, forming an 18-degree pattern. This shows a weak but visible relationship, where more rooms generally lead to higher prices. However, this pattern is not very strong, indicating that other factors also affect house prices.



Activity

Suggestions Based on Findings

- Invest in properties in areas with higher median incomes, as they tend to have higher house prices.
- Consider newer houses, but age alone may not be a significant factor in pricing.
- Larger homes with more rooms may fetch higher prices, but this effect is less significant than income.

While statistics help understand and analyse data, they also play a key role in evaluating data science models. Data science models are machine learning models created during the modelling phase of a data science project.

Model Evaluation



Simply building a model is not enough—its performance must be assessed to ensure accuracy and reliability. Model evaluation helps check whether a model's predictions are correct and trustworthy.

To do this evaluation, we use different measures to assess how well the model is performing.

For continuous values, we use **Regression Metrics**, which help evaluate models that predict numerical values, like height or temperature.

For categorical values, we use **Classification Metrics**, which evaluate models that sort data into groups, like detecting spam emails or identifying diseases.

Activity

Part 5 - Build the Model

Based on these findings till Part 4, we will create a Linear Regression model to predict house prices based on three house feature inputs i.e median income, house age and total rooms.

1 Define Features and Target

- Select the features (median income, house age, total rooms) and target variable (median house price) for the model. Assign these to 'X' and 'y' variables, respectively.

```
45
46 # Feature Selection
47 X = df[['median_income', 'housing_median_age', 'total_rooms']]
48 y = df['median_house_value']
49
50
```



2 Create and Train the Model

- Create a variable, name it as 'model'. Initialise the *LinearRegression()* model and train it using the *fit()* function with the features and target variable.

```
49
50 # Train the Linear Regression Model
51 model = LinearRegression()
52 model.fit(X, y)
53
54
```



Activity

3 Make Predictions

- Create a new *DataFrame* with hypothetical house feature values and save it in the 'new_house' variable.

```
53
54 # Make Predictions
55 new_house = pd.DataFrame({
56     'median_income': [3],
57     'housing_median_age': [20],
58     'total_rooms': [1500]
59 })
60
```

- Use the model to predict house prices based on these values in 'new_house' and save it in the 'predicted_price' variable. Print the 'predicted_price' to see the model's output.

```
59 })
60
61 predicted_price = model.predict(new_house)
62 print("Predicted House Price:", predicted_price)
63
64
```

- Execute the code to check the output.

```
median_house_value      0
ocean_proximity         0
dtype: int64
Predicted House Price: [148433.14423849]
PS D:\ICT 360>
```

Thus, we can check the predicted house price based on house features. However, sometimes the predicted values may be inaccurate. To avoid this, these predicted values should not be used directly without checking the model's efficiency.

Activity

Part 6 - Evaluate the Model

To check the model's efficiency, we can use evaluation metrics, and since the output is a continuous value, regression metrics can be used for this purpose.

1 Import Required Metrics

- Import *mean_absolute_error* and *r2_score* from *sklearn.metrics* to evaluate the model.

***sklearn.metrics*:** Provides various evaluation metrics for machine learning models, including regression and classification metrics.

***mean_absolute_error*:** Used to get Mean Absolute Error, a regression metric that measures the average absolute difference between predicted and actual values. A lower MAE (e.g., close to 0) is considered good.

***r2_score*:** Used to get R^2 Score, a regression metric that measures how well the model's predictions match the actual values. An R^2 score above 0.8 is generally considered good.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_absolute_error, r2_score
5
```

2 Make Predictions

- Create a variable 'y_pred' and use the *predict()* function to generate predictions based on the input feature 'X'.

```
62 print("Predicted House Price:", predicted_price)
63
64 # Model Evaluation
65 y_pred = model.predict(X)
66
67
```

Activity

3 Calculate Mean Absolute Error (MAE) and R² Score.

- Compute the MAE using the `mean_absolute_error()` function by passing the actual output 'y' and predicted values 'y_pred' as parameters.
- Create a variable 'r2' and pass the `r2_score()` function with 'y' (actual output from the dataset) and 'y_pred' (predictions based on input) as two parameters.

```
65 y_pred = model.predict(X)
66
67 mae = mean_absolute_error(y, y_pred)
68 r2 = r2_score(y, y_pred)
69
70
```

4 Display the Results

- Use the `print()` function to display the evaluation metrics.

```
68 r2 = r2_score(y, y_pred)
69
70 print(f"Mean Absolute Error (MAE): {mae}")
71 print(f"R2 Score: {r2}")
72
73
```

- Execute the code to display the output.

```
Predicted House Price: [148433.14423849]
Mean Absolute Error (MAE): 60065.293738606786
R2 Score: 0.5140694633254217
PS D:\ICT 360>
```

We have successfully built and evaluated a data science model for house price prediction. Currently, the model has an R² value of 0.51, indicating moderate reliability. Additionally, the Mean Absolute Error of approximately 60k suggests that the model's predictions are off by a moderate amount. Its accuracy can be improved by adding more input features or applying advanced data science techniques.

This lesson explored statistics in data science, focusing on its role, types, and model evaluation. We built a house price prediction model by collecting, preprocessing, and analysing data. Key steps included handling missing values, identifying patterns, drawing insights, and making predictions. Finally, we assessed our model using evaluation metrics.

Exercise

Fill in the Blanks.

- 1 _____ is the study of collecting, organising, analysing, and interpreting data to make informed decisions.
- 2 _____ measures how well the model's predictions match the actual values.
- 3 *Sklearn.metrics* provides various evaluation metrics for _____ models, including regression and classification metrics.

State if the Statements are True or False.

- 4 Descriptive Statistics allow us to make predictions and conclusions about a large group (population) based on a sample.
▲ _____
- 5 An R^2 score above 0.8 is generally considered bad.
▲ _____
- 6 *pd.read_csv()* function is used to read the dataset.
▲ _____
- 7 *isnull()* function combined with *sum()* can be used to check missing values in the dataset.
▲ _____

Tech Flash

- | | | |
|-------------------------------|---|--|
| Statistics | : | It is the study of collecting, organising, analysing, and interpreting data to make informed decisions. |
| Descriptive Statistics | : | It is a type of statistics that helps summarise and present data meaningfully. |
| Inferential Statistics | : | It is a type of statistics that allows one to make predictions and conclusions about a large group (population) based on a sample. |
| Central Tendency | : | It refers to measures that indicate the centre or most common value in a dataset. |
| Dispersion | : | It measures how much the data values vary or spread out from the average or mean. |
| Hypothesis Testing | : | It is a method to check if an assumption about a population is true by analysing sample data. |